

7.2 时间序列预测

7.2.1 时间序列分解

`seasonal_decompose` 是 Statsmodels 库中用于时间序列分解的一个非常有用的函数，它可以帮助我们将时间序列数据分解为趋势（Trend）、季节性（Seasonality）和残差（Residual）三个部分。下面将详细介绍 `seasonal_decompose` 的用法，并给出几个案例。

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

`seasonal_decompose` 函数的主要参数包括：

- `x`：被分解的时间序列数据，通常是一个 pandas Series 或类似数组结构的对象。
- `model`：分解模型，可以是 'additive'（加法模型）或 'multiplicative'（乘法模型），默认为 'additive'。
- `period`：时间序列的周期长度，如果 `x` 是 pandas 对象且其索引有频率，则可以省略此参数。
- `filt`：可选，用于滤除季节性成分的滤除系数，默认为 None。
- `two_sided`：滤波中使用的移动平均法，如果为 True（默认），则使用 `filt` 计算居中的移动平均线。
- `extrapolate_trend`：如果设置为 > 0 ，则考虑到许多 (+1) 最接近的点，由卷积产生的趋势将在两端外推线性最小二乘法。如果设置为 'freq'，则使用频率最近点。

`seasonal_decompose` 返回一个 `DecomposeResult` 对象，该对象包含分解出的趋势、季节性和残差成分等信息。

案例 1：饮料产品销售数据分析

假设我们有一个关于某饮料产品在上海地区2022年1月至2023年3月期间每日销售量的时间序列数据。

```
# 假设 df 是包含日期 ('date') 和销售量 ('sales') 的 pandas DataFrame
# 首先，我们需要将 'date' 列设置为索引，并确保它是时间序列索引
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)

# 使用 seasonal_decompose 进行分解
result = seasonal_decompose(df['sales'], model='additive', period=7)

# 绘制分解结果
result.plot()
plt.show()
```

在这个案例中，我们将周期设置为7天，因为饮料销售通常受到周末和工作日的影响，具有周季节性。

案例 2：月度销售数据分析

假设我们有一个关于某商品在一年中各个月份销售量的时间序列数据。

```
# 假设 df 是包含月份 ('month') 和销售量 ('sales') 的 pandas DataFrame
# 我们需要先将月份转换为时间序列索引（这里为了简化，假设月份是从1到12的连续整数）
# 在实际应用中，可能需要使用 pd.date_range 或 pd.to_datetime 与 pd.PeriodIndex 来创建时间序列索引

# 这里我们直接跳过创建时间序列索引的步骤，直接分解
# 注意：在实际应用中，你应该先确保 'month' 列是正确的时间序列索引
# result = seasonal_decompose(df['sales'], model='additive', period=12)
# 绘制分解结果
# result.plot()
# plt.show()

# 由于直接操作月份索引可能较为复杂，这里仅提供代码框架
```

注意：在这个案例中，由于缺少具体的数据和完整的索引设置，我省略了直接分解的步骤。在实际应用中，你需要先将月份数据转换为时间序列索引，并设置正确的周期（在这个例子中是12个月）。

注意：

- 在使用 `seasonal_decompose` 之前，请确保你的时间序列数据是完整的，没有缺失值。如果数据中存在缺失值，你可能需要先进行插值或删除这些缺失值。
- 选择合适的模型（加法模型或乘法模型）对于分解结果的质量至关重要。通常，如果时间序列中的趋势和季节性变化是累加的，则使用加法模型；如果它们呈现出指数增长或衰减的趋势，则使用乘法模型。

- 周期长度的选择也应该基于数据的实际情况。对于日数据，周期可能是7天（一周）；对于月数据，周期可能是12个月；等等。

7.2.2 ARIMA模型

在Python的 `statsmodels` 库中，ARIMA（自回归积分滑动平均模型）是一种常用的时间序列预测模型。ARIMA模型结合了自回归（AR）、差分（I）和移动平均（MA）三个部分来预测未来值。以下是如何在 `statsmodels` 中使用ARIMA模型进行时间序列预测的详细步骤和代码案例。

首先，你需要导入 `pandas` 用于数据处理，`matplotlib.pyplot` 用于绘图，以及 `statsmodels.tsa.arima.model.ARIMA` 用于建立ARIMA模型。

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
import numpy as np
```

加载时间序列数据，并将其转换为适合ARIMA模型的形式。数据需要是平稳序列，所以，在进行时间序列预测之前要进行平稳性检验。这里我们一般使用ADF单位根检验。

```
# 假设你有一个CSV文件，其中包含时间序列数据
df = pd.read_csv('time_series_data.csv', index_col='Date', parse_dates=True)
ts = df['Value'] # 假设时间序列数据在 'Value' 列

# 检查平稳性（ADF测试）
result = adfuller(ts)
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])

# 如果p-value很大，则时间序列可能不是平稳的，需要进行差分
ts_diff = ts.diff().dropna()

# 再次检查差分后的平稳性（可选）
result_diff = adfuller(ts_diff)
print('ADF Statistic: %f' % result_diff[0])
print('p-value: %f' % result_diff[1])
```

一旦你的时间序列是平稳的（或你认为它足够接近平稳），你就可以使用ARIMA模型进行拟合了。你需要指定模型的参数(p, d, q)，其中p是自回归项的阶数，d是差分阶数（已经通过步骤2的差分操作隐含指

定)， q 是移动平均项的阶数。

```
# 假设我们已经通过某种方式（如ACF和PACF图）选择了p, d, q
# 注意：这里的d=1是因为我们在步骤2中已经进行了一次差分
p = 1
d = 1
q = 1

# 拟合ARIMA模型
model = ARIMA(ts, order=(p, d, q))
fit_model = model.fit()

# 打印摘要信息
print(fit_model.summary())
```

使用拟合好的模型进行未来值的预测。

```
# 预测未来5个时间点的值
forecast = fit_model.forecast(steps=5)
print(forecast)

# 绘制原始数据和预测数据
plt.figure(figsize=(10, 5))
plt.plot(ts.index, ts, label='Original')
plt.plot(ts.index[-1:] + pd.DateOffset(days=1):ts.index[-1:] + pd.DateOffset(days=6), forecast,
plt.legend()
plt.show()
```

注意：在上面的预测代码中，我使用了 `ts.index[-1:] + pd.DateOffset(days=x)` 来生成未来时间点的索引，其中 x 是从1到5的整数。这是为了将预测结果绘制在正确的日期上。但是，这假设了时间序列是日数据，并且每天都有一个观测值。如果你的数据频率不同（如月度、季度等），你需要相应地调整索引的生成方式。

注意：

- 在实际应用中，选择正确的ARIMA模型参数 (p, d, q) 是关键。这通常涉及到查看时间序列的ACF（自相关函数）和PACF（偏自相关函数）图，或者使用网格搜索等优化方法来找到最佳参数。
- 有时，对数据进行季节性差分或转换（如对数变换）可能是必要的，以进一步改善模型的性能。
- 始终关注模型的诊断统计量和残差图，以确保模型没有遗漏重要的信息或存在其他问题。